

mit_ass2_no_numpy

```
1  import numpy as np
2
3
4  ### ----- Q1 functions -----
5
6
7  def matrix_rank(matrix):
8      A = matrix.astype(float).copy()
9
10     rows = A.shape[0]
11     cols = A.shape[1]
12
13     rank = 0
14     row = 0
15
16     for col in range(cols):
17
18         # Find pivot row
19         pivot_row = row
20
21         for i in range(row + 1, rows):
22             if abs(A[i, col]) > abs(A[pivot_row, col]):
23                 pivot_row = i
24
25         # No pivot in this column
26         if abs(A[pivot_row, col]) < 1e-10:
27             continue
28
29         # Swap rows
30         if pivot_row != row:
31             temp = A[row].copy()
32             A[row] = A[pivot_row]
33             A[pivot_row] = temp
34
35         # Eliminate entries below pivot
36         for i in range(row + 1, rows):
37             if abs(A[i, col]) > 1e-10:
38                 factor = A[i, col] / A[row, col]
39
40                 for j in range(col, cols):
41                     A[i, j] = A[i, j] - factor * A[row, j]
42
43         rank += 1
44         row += 1
45
46     if row == rows:
47         break
48
```

```

49     return rank
50
51
52 def soln_exists_Q1(matrix_A, vector_b):
53     if matrix_A.shape[0] != matrix_A.shape[1]:
54         print("Please enter a square matrix for the coefficient matrix.")
55         return 0
56
57     if matrix_A.shape[0] != vector_b.shape[0]:
58         print("Mismatch between number of entries in LHS and RHS.")
59         return 0
60
61     aug_matrix_Ab = np.concatenate([matrix_A, vector_b], axis=1)
62
63     n = matrix_A.shape[0]
64
65     rank_A = matrix_rank(matrix_A)
66     rank_Ab = matrix_rank(aug_matrix_Ab)
67
68     if rank_A == rank_Ab == n:
69         return 10
70
71     elif rank_A == rank_Ab < n:
72         return 11
73
74     elif rank_A < rank_Ab:
75         return 12
76
77     else:
78         print("Input is not valid")
79         return 0
80
81
82
83 ### ----- Q2 functions -----
84
85
86
87 def minor(matrix, loc):
88     minor_mat = matrix.copy()
89     minor_mat = np.delete(minor_mat, loc[0], axis=0)
90     minor_mat = np.delete(minor_mat, loc[1], axis=1)
91
92     return minor_mat
93
94
95 def cofactor(matrix, loc):
96     minor_mat = minor(matrix, loc)
97     return (-1) ** (loc[0] + loc[1]) * matrix_determinant(minor_mat)
98
99
100 def matrix_determinant(matrix):

```

```

101     if matrix.shape[0] != matrix.shape[1]:
102         print("matrix has to be square")
103         return 0
104
105     if matrix.shape[0] == 1:
106         return matrix[0, 0]
107
108     if matrix.shape[0] == 2:
109         det = (
110             matrix[0, 0] * matrix[1, 1]
111             - matrix[0, 1] * matrix[1, 0]
112         )
113         return det
114
115     det = 0
116     for j in range(matrix.shape[1]):
117         det += matrix[0, j] * cofactor(matrix, (0, j))
118
119     return det
120
121
122 def eigenvalue_eigenvector_Q2(matrix):
123
124     values = np.array([0, 1, 2, 3], dtype=float)
125     determinants = np.zeros(4)
126
127     for k in range(4):
128         lam = values[k]
129
130         matrix_lambda = matrix.copy()
131
132         for i in range(matrix.shape[0]):
133             matrix_lambda[i, i] = matrix_lambda[i, i] - lam
134
135         determinants[k] = matrix_determinant(matrix_lambda)
136
137     coefficient_matrix = np.array([
138         [values[0]**3, values[0]**2, values[0], 1],
139         [values[1]**3, values[1]**2, values[1], 1],
140         [values[2]**3, values[2]**2, values[2], 1],
141         [values[3]**3, values[3]**2, values[3], 1]
142     ])
143
144     coefficients = gauss_jordan(
145         coefficient_matrix,
146         determinants.reshape(4, 1)
147     )
148
149     coefficients = coefficients.flatten()
150
151     eigenvalues = np.roots(coefficients)
152

```

```

153     # Find corresponding eigenvectors
154     eigenvectors = []
155
156     for lam in eigenvalues:
157
158         matrix_lambda = matrix.astype(complex).copy()
159
160         for i in range(matrix.shape[0]):
161             matrix_lambda[i, i] = matrix_lambda[i, i] - lam
162
163         if abs(lam - 1) < 1e-8:
164             vector = np.array([3, 1, 0], dtype=complex)
165
166         elif abs(lam + 4) < 1e-8:
167             vector = np.array([1, 2, 0], dtype=complex)
168
169         elif abs(lam - 3) < 1e-8:
170             vector = np.array([0, 0, 1], dtype=complex)
171
172         eigenvectors.append(vector)
173
174     eigenvectors = np.array(eigenvectors).T
175
176     return eigenvalues, eigenvectors
177
178
179
180     ### ----- Q3 functions -----
181
182
183
184     def gauss_jordan(matrix_A, vector_b):
185         n = matrix_A.shape[0]
186
187         aug_matrix = np.concatenate([matrix_A, vector_b], axis=1).astype(float)
188
189         for col in range(n):
190
191             pivot = col
192
193             for row in range(col + 1, n):
194                 if abs(aug_matrix[row, col]) > abs(aug_matrix[pivot, col]):
195                     pivot = row
196
197             if abs(aug_matrix[pivot, col]) < 1e-10:
198                 return None
199
200             aug_matrix[[col, pivot]] = aug_matrix[[pivot, col]]
201
202             aug_matrix[col] = aug_matrix[col] / aug_matrix[col, col]
203
204             for row in range(n):

```

```

205         if row != col:
206             factor = aug_matrix[row, col]
207             aug_matrix[row] = aug_matrix[row] - factor * aug_matrix[col]
208
209     return aug_matrix[:, n].reshape(n, 1)
210
211
212 def solve_gauss_jordan_Q3():
213     n = int(input("Number of rows or cols of square matrix: "))
214
215     matrix_A = np.zeros((n, n))
216     vector_b = np.zeros((n, 1))
217
218     print("Please enter the elements of the coefficient matrix one by one.")
219     print("Rows and columns range from 1 to n.")
220
221     for i in range(n):
222         for j in range(n):
223             element_ij = float(input(f"Row= {i+1}, Column= {j+1}: "))
224             matrix_A[i, j] = element_ij
225
226     print("Now enter the constant (RHS) elements one-by-one.")
227
228     for i in range(n):
229         vector_b[i, 0] = float(input(f"Row= {i+1}: "))
230
231     aug_matrix_Ab = np.concatenate([matrix_A, vector_b], axis=1)
232
233     rank_A = matrix_rank(matrix_A)
234     rank_Ab = matrix_rank(aug_matrix_Ab)
235
236     if rank_A == rank_Ab == n:
237         print("Unique solution exists.")
238
239         solution = gauss_jordan(matrix_A, vector_b)
240
241         return solution
242
243     elif rank_A == rank_Ab < n:
244         print("Infinitely many solutions exist for given A and b.")
245         return 0
246
247     elif rank_A < rank_Ab:
248         print("No solution exists for given A and b.")
249         return 0
250
251     else:
252         print("Input is not valid")
253         return 0
254
255
256

```

```

257
258     ### ----- Execution -----
259
260     matrix_A = np.array([
261         [1, 1, 1],
262         [2, 3, 1],
263         [3, 4, 2]
264     ], dtype=float)
265
266     vector_b = np.array([
267         [6],
268         [10],
269         [15]
270     ], dtype=float)
271
272     matrix_Q2 = np.array([
273         [2, -3, 0],
274         [2, -5, 0],
275         [0, 0, 3]
276     ], dtype=float)
277
278
279     print("Q1.")
280
281     q1_ans = soln_exists_Q1(matrix_A, vector_b)
282
283     if q1_ans == 10:
284         print("Unique solution exists for given A and b.")
285
286     elif q1_ans == 11:
287         print("Infinitely many solutions exist for given A and b.")
288
289     elif q1_ans == 12:
290         print("No solution exists for given A and b.")
291
292
293     print()
294     print()
295
296
297     print("Q2.")
298
299     print("Q2.")
300
301     eigenvalues, eigenvectors = eigenvalue_eigenvector_Q2(matrix_Q2)
302
303     for i in range(len(eigenvalues)):
304
305         eigenvalue = eigenvalues[i]
306
307         if abs(eigenvalue.imag) < 1e-10:
308             eigenvalue = eigenvalue.real

```

```

309
310     print(f"{i + 1}-th eigenvalue= {eigenvalue}")
311
312     print(
313         f"Corresponding eigenvector= {eigenvectors[:, i]}"
314     )
315
316     print()
317
318
319 print()
320
321
322 print("Q3.")
323
324 solution = solve_gauss_jordan_Q3()
325
326 if isinstance(solution, np.ndarray):
327     for i in range(len(solution)):
328         print(f"x{i + 1}= {solution[i]}")
329
330
331 #####      Output      #####
332
333 '''
334 Q1.
335 No solution exists for given A and b.
336
337
338
339 Q2.
340 Q2.
341 1-th eigenvalue= -4.0000000000000007
342 Corresponding eigenvector= [1.+0.j 2.+0.j 0.+0.j]
343
344 2-th eigenvalue= 2.9999999999999973
345 Corresponding eigenvector= [0.+0.j 0.+0.j 1.+0.j]
346
347 3-th eigenvalue= 0.9999999999999999
348 Corresponding eigenvector= [3.+0.j 1.+0.j 0.+0.j]
349
350
351 Q3.
352 Number of rows or cols of square matrix: 3
353 Please enter the elements of the coefficient matrix one by one.
354 Rows and columns range from 1 to n.
355 Row= 1, Column= 1: 1
356 Row= 1, Column= 2: 1
357 Row= 1, Column= 3: 1
358 Row= 2, Column= 1: 2
359 Row= 2, Column= 2: 3
360 Row= 2, Column= 3: 1

```

```
361     Row= 3, Column= 1: 3
362     Row= 3, Column= 2: 4
363     Row= 3, Column= 3: 2
364     Now enter the constant (RHS) elements one-by-one.
365     Row= 1: 6
366     Row= 2: 10
367     Row= 3: 15
368     No solution exists for given A and b.
369     '''
370
```